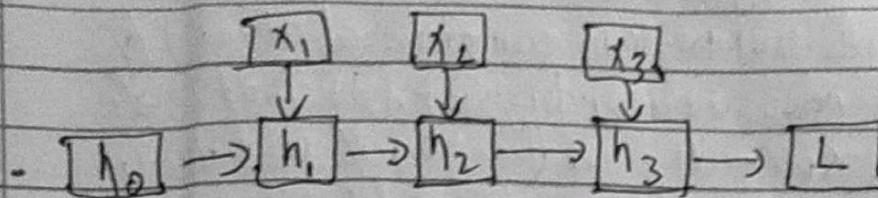


Assignment 4: RNN

ZOSMAN BUAKATI

Date: _____

Page: _____



① Forward Pass:

At each step the RNN takes the current input x_t and previous hidden state h_{t-1} , and produces a new hidden state through the same function f (typically a linear map followed by $\tanh$): $f(x_t, h_{t-1}) = \tanh(W \cdot x_t + U \cdot h_{t-1} + b)$.

Starting from an initial state h_0 (usually zeros), the network processes x_1, x_2, x_3 one at a time, each step's output feeding into the next. The diagram above is that unrolled graph: it turns the recurrent (looped) definition into a feedforward chain, which is what actually lets you apply ordinary backprop to it.

Every arrow is a real data dependency: h_3 depends on h_2 and x_2 and so on. The final scalar L depends only on h_3 but directly, but, transitively, on everything upstream.

(2) hidden state dynamics

h_t is the network's compressed memory of everything it has seen up to step t . It isn't just a function of x_t - it's a function of x_t & h_{t-1} , so by induction $h_t = f(x_t, f(x_{t-1}, f(x_{t-2}, \dots)))$.

Each step blends new information (x_t) with an existing summary of the past (h_{t-1}) into an updated summary. This is what lets RNN handle variable-length sequences with a fixed-sized state: context doesn't get stored explicitly per time step, it gets folded in & carried forward, so h_t implicitly "contains" traces of x_1 and x_2 even though it only takes x_3 & h_2 as direct arguments.

(3) Backpropagation Through Time (BPTT)

Because L is only a direct function of h_3 , computing $\partial L / \partial h_t$ requires walking back through the chain $h_3 \rightarrow h_2 \rightarrow h_1$, applying the chain rule at each link - exactly the way you'd backprop through a deep feedforward network of 3 layers except here every "layer" reuses the same weights. It's called BPTT because the depth being backpropagated through isn't network layers stacked in space, it's the same layer applied repeatedly across time steps - so gradients literally flow backward along the time axis.

④ Chain rule derivation.

Since $h_3 = f(x_3, h_2)$ & $h_2 = f(x_2, h_1)$ the dependency chain from L to h_1 is $L \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$.

Applying the multivariable chain rule:

$$\partial L / \partial h_2 = (\partial L / \partial h_3) \cdot (\partial h_3 / \partial h_2)$$

$$\begin{aligned} \partial L / \partial h_1 &= (\partial L / \partial h_3) \cdot (\partial h_3 / \partial h_2) \cdot (\partial h_2 / \partial h_1) \\ &= (\partial L / \partial h_3) \cdot (\partial h_3 / \partial h_2) \cdot (\partial h_2 / \partial h_1) \end{aligned}$$

So the final expression is:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1}$$

Each Jacobian $\partial h_t / \partial h_{t-1}$ has the form $(1 - h_t^2) \cdot W_{hh}$ (the derivative of $\tanh$ times the recurrent weight matrix), so more explicitly:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_3} \cdot \text{diag}(1 - h_3^2) \cdot W_{hh} \cdot \text{diag}(1 - h_2^2) \cdot W_{hh}$$

Notice W_{hh} appears twice, multiplied; this product structure is exactly what causes vanishing/exploding gradients over longer sequences.

(5)

Why intermediate activation must be stored

Every term in that chain-rule product: $\partial h_3 / \partial h_2$, $\partial h_2 / \partial h_1$, and so on — is evaluated at the specific values h_1 , h_2 , h_3 computed during the forward pass (e.g. $\text{diag}(1 - h_2^2)$ needs the actual numeric h_2). Since the backward pass can only run after the full forward pass finishes, every hidden state has to be cached in memory during the forward pass so it's available when its local gradient is computed later. This is exactly why BPTT's memory cost scales linearly with sequence length T . The dominant memory cost is ~~one~~ motivation for truncated BPTT or gradient checkpointing.

(6)

Parameter sharing

The same W_{xh} , W_{hh} , and b are used at every time step rather than having separate weights per step. This is what makes the model translation-invariant across time (it can generalize a pattern learned at step 2 to step 5) and keeps the parameter count independent of sequence length. For gradients, sharing means the same weight matrix appears in the computation graph at every time step, so its total gradient is a sum over all the time steps' contributions: $\partial L / \partial W_{hh} = \sum_t (\partial L / \partial h_t) \cdot (\partial h_t / \partial W_{hh})$, accumulated across $t = 1 \dots T$ before the

single shared update is applied. Parameter sharing is what turns "gradient at each time step" into "gradient accumulation across time".

⑦ Vanishing & exploding gradients.

As derived in ④, computing ~~across~~ gradient across many steps means multiplying many Jacobian terms $\text{diag}(1 - h_t^L) \cdot W \cdot h_t$ together, one per time step. This is essentially a repeated matrix product:

- If the dominant eigenvalues of $W \cdot h_t$ (scaled by tanh derivatives, which is < 1) are consistently < 1 , the product shrinks geometrically with sequence length - gradients vanish, and early time steps stop receiving any meaningful learning signal.
- If those eigenvalues are consistently > 1 , the product grows geometrically - gradients explode, causing unstable, huge weight updates (often producing NaNs).

⑧ Implementation:

Note: checkout sibling py script & README.md for further explanation.